URECAMatch: Getting Started

SC2006 · SCEF Group 4 · Lab manual for team members · 15 September 2026

Do these steps once on your laptop. Each step says what to type and why. Total time: about 30 minutes. Commands go in a terminal: on Mac search "Terminal", on Windows search "PowerShell".

1 Install the tools

Type these four commands one at a time to check what you already have:

```
node –v
python3 ––version
uv ––version
git ––version
```

If a command prints a version, you have it. If it says **not found**, install it:

TOOL	WHAT IT IS FOR	WHERE TO GET IT
Node.js 20 or newer	runs the web app	nodejs.org, LTS installer
Python 3.12	runs the backend	python.org. On Windows tick "Add to PATH"
uv	installs Python packages for us	Mac: `brew install uv`. Windows: `powershell –c "irm https://astral.sh/uv/install.ps1 | iex"`
Git	shares code with the team	git-scm.com
VS Code	the editor	code.visualstudio.com

Close and reopen the terminal after installing, then run the four commands again.

2 Tell Git your name

Git writes your name on every change, so the team knows who did what.

```
git config ––global user.name "Your Name"
git config ––global user.email "you@e.ntu.edu.sg"
```

3 Choose a folder

This creates a Projects folder in your home directory and moves into it.

```
mkdir –p ~/Projects
cd ~/Projects
```

4 Download the project (clone)

Cloning copies the whole repo from GitHub to your laptop, including its history.

```
git clone https://github.com/softwarelab3/2006-SCEF-A4.git
```

If GitHub asks you to log in, use your GitHub username and a personal access token: GitHub → Settings → Developer settings → Personal access tokens → Generate new token → tick `repo`. You must be a collaborator on the repo; ask Robin if the clone is refused.

5 Open it in VS Code

```
cd 2006-SCEF-A4/ureca-match-app
code .
```

You will see four folders: **web** (the screens), **api** (the backend), **supabase** (the database), **docs** (the plan and the contributing guide).

6 Run the web app

The web app works on its own with sample data. Nothing else needs to run. In VS Code open a terminal (menu Terminal → New Terminal) and type:

```
cd web
cp .env.example .env.local
npm install
npm run dev
```

- `cp .env.example .env.local` copies the settings file. It says "use sample data".
- `npm install` downloads the packages the app needs. 1 to 2 minutes, first time only.
- `npm run dev` starts the app.

Open **http://localhost:3000** in your browser. Click a demo button (Student, Supervisor, Admin) to sign in and click around. Everything you see is the skeleton you will fill in. Keep this terminal running while you work; press Ctrl+C to stop it.

7 Run the backend and its tests

Open a second terminal (Terminal → New Terminal).

```
cd api
cp .env.example .env
uv sync
uv run pytest
```

- `uv sync` installs the Python packages, first time only.
- `uv run pytest` runs the tests. You should see **44 passed**. This proves your setup is correct.

Now start the backend:

```
uv run fastapi dev app/main.py
```

Open **http://localhost:8000/docs**. This page lists every API endpoint and lets you call them. For a protected endpoint click **Authorize** and enter a demo user id such as `stu-1`, `sup-1` or `adm-1`.

> Demo accounts: any seeded e-mail with password `password`. E-mail verification code `000000`. The clock is fixed at 7 Sep 2026, inside Round 2's application window, so every screen has data.

8 Read your part of the plan (20 minutes)

- Open `LAB2/LAB 2 (SRS).pdf`, section 4. Find your features. Each rule has an ID like `REQ-REG-1`. We use these IDs everywhere: in code comments, test names, commit messages and pull requests.
- Open `ureca-match-app/docs/DEV-PLAN.md`. Find your slice (A to E). It lists your endpoints, the logic to build and the tests to write.
- Open `ureca-match-app/docs/CONTRIBUTING.md`, Part C. It explains what each folder and file is for.

SLICE	BACKEND FILES (API/APP/ROUTERS)	SCREENS (WEB/APP)
A Accounts and profile	auth.py, profile.py, users.py	(auth)/*, (student)/profile, (admin)/users
B Projects and proposals	listings.py, proposals.py	(student)/browse, projects/[id], propose, (supervisor)/projects/*, proposals
C Applications and rankings	applications.py, rankings.py	(student)/apply/[id], my-applications, (supervisor)/projects/[id]/applicants
D Cycle, rounds, matching	cycles.py, matching.py, services/matching/	(admin)/rounds/*, (supervisor)/approvals, offer card in my-applications
E Platform	notifications.py, services/notify.py, services/audit.py	notifications, (admin)/dashboard, components/ui, CI, deploy

9 Find your work in the code

Every place that needs your work is marked with a comment. From the `ureca-match-app` folder:

```
grep -rn "TODO(slice A)" api/app web/app
```

Replace **A** with your slice letter. Each line printed is a file and line number where your code goes. On Windows PowerShell use: `Select-String -Path api\app\*,web\app\* -Pattern "TODO(slice A)" -Recurse`

10 Do one practice change

Before real work, prove the whole flow works for you. From the repo root (`2006-SCEF-A4`):

```
git checkout main
git pull
git checkout -b slice-a/hello
```

- `git pull` gets the latest code from GitHub.
- `git checkout -b` creates your own branch. Your changes live here until they are reviewed.

Open your router file in `ureca-match-app/api/app/routers/` and add one comment line at the top, for example `# Slice A owner: Ian`. Then:

```
git add -A
git commit -m "Slice A: owner note"
git push -u origin slice-a/hello
```

- `git add -A` stages your change.
- `git commit -m` saves a checkpoint with a message.
- `git push` uploads your branch to GitHub.

11 Open a pull request

Go to the repo on GitHub. A yellow banner says your branch had recent pushes. Click **Compare & pull request**.

- Title: `Slice A: owner note`
- Reviewers: one teammate from a different slice
- Click **Create pull request**

Two checks run automatically, **ci / api** and **ci / web**. Wait for both to turn green. When your reviewer approves, click **Squash and merge**, then **Delete branch**.

You have now done the full loop once. Real features follow steps 10 and 11 again, with the worked example in `docs/CONTRIBUTING.md` Part D showing how to write the test first, implement, and check the screen.

12 If something breaks

SYMPTOM	WHAT TO DO
`npm run dev` will not start	Check `node -v` is 20 or higher. Delete `node_modules` and run `npm install` again.
`uv run pytest` says module not found	Run `uv sync` inside `api`.
A screen is blank	Press F12 in the browser, open the Console tab. The error says which rule blocked the action.
A check on GitHub is red	Click **Details**. It runs the same commands as steps 6 and 7. Run them locally to see the error.
`git pull` reports a conflict	`git status` shows the file. Fix the lines between `<<<<<<<` and `>>>>>>>`, then `git add` and `git commit`.
Stuck for more than 15 minutes	Paste the exact error in the group chat.

Repo: github.com/softwarelab3/2006-SCEF-A4 · Full guide: ureca-match-app/docs/CONTRIBUTING.md · Plan: ureca-match-app/docs/DEV-PLAN.md